INSOMNIA

Clarity before commitment.

Product documentation & backend readiness

An account-based workspace that turns IOMETE buying and migration questions into requirements, sourced guidance, test plans and decision-ready evidence.

9	23	42
server tool engines	workspace routes	passing automated tests

Edition 1.0 / November 1, 2026

Current status: the implemented evaluation workflows have a working backend. External cluster execution, live research, model integrations and production-launch work remain separate.

Independent IOMETE-track project. This guide does not claim an independently proven market ranking.

Why Insomnia exists

PRODUCT PURPOSE / pages 3-4 tools / page 5 workflow / page 6 backend / pages 7-8 operations and readiness / page 9 references

A data platform can sound compelling while leaving the buying decision unresolved. Buyers still need to know which workloads fit, what the infrastructure needs, what the costs mean, and what evidence will justify a pilot or purchase. Insomnia organizes that work around one company evaluation.

The job it completes

Define a workload, document the buyer's success criteria, map needs to official product information, assign tests, import observations, and export a brief that separates documented facts from assumptions and unknowns.

Design strength	Practical value
Connected company context	Tools use the selected evaluation and save their outputs with its requirements and source links.
Traceable rules and calculations	Cost totals, prerequisite checks and benchmark gates are explicit rather than unexplained scores.
Evidence discipline	Unsupported security claims stay unknown. A capability description is not treated as customer assurance.
Report-level benchmark gates	Critical reports, correctness, concurrency and missing samples remain visible even when aggregate results look good.
Real persistence and accounts	Server-side validation, SQLite storage, authenticated sessions and account isolation support work across visits.
Reviewable handoff	Markdown briefs, saved tool snapshots, JSON backups and audit records make the evaluation easier to inspect.

What makes it a strong choice

Insomnia combines the six buying-journey ideas from the track brief with workload, deployment and migration review. Its strongest differentiator is a complete, traceable evaluation workflow. These are design strengths supported by implementation; usability, commercial impact and superiority over competing tools have not been independently benchmarked.

Who it serves

Buyers define success; sales engineers structure demonstrations and pilots; security leads identify assurance gaps. The perspective selector changes the interface context. It does not grant additional security permissions or establish shared-team membership.

Understand, qualify, estimate

FEATURE GUIDE / every tool is executed and saved through the authenticated backend

01 Product guide

Inputs: A question and stakeholder role.
Core logic: Matches the question to a curated official-documentation snapshot and selects role-specific follow-up questions.
Output: A source-linked explanation, relevant topics and the next questions to ask.
Boundary: This is deterministic retrieval, not live web research or a connected language model. Unsupported topics are reported.

02 Use-case finder

Inputs: Needs such as customer-controlled infrastructure, analytics, Oracle modernization, open tables and governance.
Core logic: Applies visible, equal-weight rules; surfaces transactional-point-lookup concerns and a fully managed SaaS mismatch.
Output: A shortlist of evaluation use cases, constraints and a proposed next step.
Boundary: The result is a screening aid, not a certification that a particular workload will fit.

03 Deployment readiness

Inputs: Environment, Kubernetes, storage, PostgreSQL, ingress, CPU/RAM and an operating owner.
Core logic: Compares supplied values with documented installation floors; marks missing values unknown and checks offline delivery for air-gapped setups.
Output: A prerequisite gap list with explicit confirmation and production-sizing questions.
Boundary: It does not inspect a cluster. Version-specific requirements and platform exceptions require confirmation.

04 Business case

Inputs: USD license inputs, current cost, infrastructure, operations, services, migration, overlap and time horizon.
Core logic: Calculates recurring and transition costs, applies the Enterprise list-estimate minimum or entered quote, and derives savings and simplified payback.
Output: A line-by-line, exportable cost case showing the supplied assumptions.
Boundary: No taxes, inflation, discount rates, workload capacity or productivity gains are inferred. Pricing is a dated snapshot, not a quote.

How outputs stay reviewable

Each successful run captures inputs, result sections, source topics and a server timestamp. Results can be revisited, exported individually, or included in the company brief. Rerun after changing the evaluation or measurements; a saved output is a historical snapshot.

Prepare, test, decide

FEATURE GUIDE / no unexecuted test is presented as a measured success

05 Call preparation

Inputs: Evaluation context, stakeholder, duration, supplied company facts and the desired decision.
Core logic: Builds a timed agenda, role-specific questions and a list of unresolved tests.
Output: A practical preparation brief tied to the buyer's evaluation.
Boundary: Company facts are supplied by the user; source URLs are not automatically researched or verified.

06 Demo planner

Inputs: Industry, primary concern and demo duration.
Core logic: Selects an industry-specific synthetic schema, scenario and roles; adds focus-specific checkpoints and timed stages.
Output: A runbook for banking, manufacturing, retail or healthcare.
Boundary: The runbook does not execute a demo or claim customer outcomes.

07 Security questionnaire

Inputs: Questions entered as lines or imported from a one-column CSV.
Core logic: Conservatively matches public capability topics; flags compound questions and unsupported assurance requests; calculates the unknown rate.
Output: Review drafts with sources, unknowns and reasons for further investigation.
Boundary: It does not certify compliance or infer customer-specific encryption, recovery, contractual or audit assurances.

08 Benchmark review

Inputs: Report inventory, imported measurements, required concurrency and minimum sample count.
Core logic: Computes nearest-rank p95 per report; checks sample coverage, concurrency and correctness; highlights critical report failures.
Output: A report-level pass, failure or insufficient-evidence assessment.
Boundary: Measurements are user-provided. Insomnia does not run SQL against an IOMETE cluster.

09 Migration review

Inputs: SQL, critical report targets, and CDC/reconciliation confirmations.
Core logic: Scans SQL for selected Oracle-to-Spark review flags after removing comments and quoted literals; surfaces migration gates.
Output: A review checklist for syntax, correctness, concurrency, source permissions and cutover.
Boundary: It is a heuristic reviewer, not a SQL compiler, automatic translator or performance predictor.

Use Insomnia, end to end

USER GUIDE / the new account starts empty; sample data is optional

1. Create an account or sign in

Use the Insomnia server account form. Passwords must be 12-128 characters. No IOMETE account or external API key is required for the current toolset.

2. Create an evaluation

Enter the company, workload name, platform, owner, target date and success criteria. The backend creates linked proposed acceptance tests with unknown results.

3. Build the workload inventory

Add reports manually or import a CSV. Baselines and targets are buyer-provided seconds. Identify business-critical reports and agree the concurrency target.

4. Run the relevant tools

Select the company evaluation and supply its facts. Run guidance, fit, deployment, cost, call, demo or security tools. Inputs and outputs are saved after server acknowledgment.

5. Record and review evidence

Assign test owners and criteria. Import actual measurements for existing report names, run benchmark review, and record observations. The app does not silently mark acceptance tests passed.

6. Export the decision brief

Download company plans or tool reports as Markdown. Inspect saved snapshots, server audit and deadline blockers. Use a JSON backup before replacing or clearing data.

CSV headers

```
Workload inventory:
report,baseline,target,critical

Benchmark observations:
report,seconds,concurrency,correct

Security questionnaire:
question
```

CSV values must meet the relevant validation rules. Use true/false for critical and correct fields; quote cells containing commas. Templates can be downloaded from the matching screen. No example measurement is inserted as a real result.

A working backend

TECHNICAL GUIDE / Node.js + SQLite / one origin for UI and API

Browser form -> authenticated JSON API -> validation and tool engine -> SQLite transaction -> new revision and audit event -> confirmed UI state. The frontend does not mark a write saved or clear a submitted form before server confirmation.

API area	Implemented responsibility
GET /api/health	Reports available engines and integration capability status.
POST /api/auth/signup, /login, /logout	Creates accounts, verifies passwords, establishes sessions and revokes logout sessions.
GET /api/session, /workspace	Resolves the signed-in account and reads its saved workspace and revision.
PUT /api/workspace	Validates evaluation, requirement, workload, test, settings and deletion changes; enforces revisions and immutable tool outputs.
POST /api/tools/run	Executes all nine tool engines against saved account data and persists authoritative results.
POST /api/import	Parses workload, measurement and security-question CSV on the server.
POST /api/workspace/restore, /reset	Validates backups, marks imported snapshots and handles explicit sample or empty resets.
GET /api/export	Creates company briefs, saved report downloads and workspace JSON backups from persisted records.
GET /api/insights, /audit	Computes dashboard metrics, weekly activity, readiness and blockers; reads account-isolated server audit records.

Data integrity and concurrency

Each account owns its workspace. Saves compare the client revision with the current database revision, then commit data and an audit entry in a transaction. Stale revisions are rejected, the latest data is reloaded, and the user can review and retry. Ordinary client edits cannot create or rewrite generated tool results.

Account and session controls

Accounts use salted scrypt password hashes, opaque HTTP-only session cookies, SameSite=Strict, CSRF and origin checks, and authentication rate limiting. Logout revokes the server session. The database persists across server restarts and is excluded from Git.

Run, save, recover

OPERATIONS / current implementation is a single-server application

Development

```
Node.js 24.11 or later
npm ci
npm run dev
```

Open the printed URL, normally http://127.0.0.1:5181/. The account screen is /portal/#auth. Use the full-stack server, not Live Server or a standalone Vite page.

Production serving

```
npm run build
npm start
```

The production server serves the built frontend and the same authenticated API. The default listener is local-only. Remote hosting needs HTTPS, secure cookies, a configured public origin, a durable database volume and deployment operations.

Concern	Current behavior
Persistent storage	.data/insomnia.sqlite. Database and environment files are private and ignored by Git.
Backup and restore	JSON contains workspace records, not account passwords or sessions. Restored report snapshots are explicitly marked imported.
Previous browser data	Use Settings to import previous browser data. For an old app on another port, export its JSON there and restore it into the server account.
Conflict or connection failure	Inputs remain available. Retry connection or reload the latest revision before deliberately resubmitting a change.
Limits	Requests: 5 MB. Saved workspace: 4 MB. CSV and row limits are enforced. Snapshots retain the latest 2,000 events; database audit is separate.
Account scope	Each account has its own workspace. The role selector is a perspective setting, not shared-team permissions.

Configuration is described in .env.example and BACKEND.md. A workspace JSON export is not a full database backup. Use a suitable SQLite backup procedure or stop the server before copying database files. Node's bundled SQLite API currently emits an experimental-feature notice.

What is done. What is next.

IMPLEMENTATION STATUS / assessed against the current repository

Status	Scope
Implemented and tested	All nine tool engines execute through authenticated backend endpoints.
Implemented and tested	Account creation, password sign-in, session logout, per-account storage, revisions and server validation.
Implemented and tested	CSV imports, persisted outputs, exports, settings, custom tests, metadata edits, resets, backups and server audit.
Implemented and tested	Dashboard counts, weekly activity, completion and deadline blockers use persisted records and server calculations.
Pending verification	A complete real-browser review of interactions, responsive layout, accessibility, video behavior and failure recovery.
Not connected	IOMETE cluster access, direct SQL execution and automatic benchmark collection.
Not connected	Live company intelligence, connected language-model generation and automatic documentation refresh.
Not implemented	Team invitations, shared organization workspaces, granular team permissions, enterprise SSO/OAuth, email verification and password-reset delivery.
Required before broad production use	Deployment monitoring, restore drills, load testing, independent security review, operational ownership and multi-server strategy if needed.

Verification evidence

The current production frontend build passes. The test suite reports 42 passing tests, including HTTP calls to all nine engines, account isolation, CSRF, persistence after restart, stale revision rejection, CSV validation, exports and logout. Server-render checks exercise 23 routes with populated and empty workspaces. These checks do not replace browser interaction tests, a penetration test or a load test.

Is the backend done?

The backend is implemented for the current evaluation-workspace scope. It is not accurate to call every possible integration or production concern complete. The next highest-value work is real-browser end-to-end validation, followed by the specific external integration required by the intended users.

Sources and next steps

REFERENCE / official product facts are separate from Insomnia implementation claims

IOMETE source snapshot reviewed November 1, 2026. Features, pricing, entitlements and requirements may change; validate current terms and the applicable product version before a customer decision.

IOMETE deployment options

https://iomete.com/product/deployment

On-premises installation requirements

https://iomete.com/resources/deployment/on-prem/install

Technical architecture

https://iomete.com/resources/blog/iomete-core-components-technical-architecture

Oracle-to-Spark migration experience

https://iomete.com/resources/blog/oracle-to-spark-migration

Pricing and commercial plan details

https://iomete.com/pricing

Software license and operating responsibility

https://iomete.com/policies/software-license-agreement

Insomnia source repository

https://github.com/harsh-dixit21/Insomnia

Repository documentation

README.md covers setup and the feature guide. BACKEND.md documents endpoints and server behavior. tests/ contains the algorithm and HTTP integration checks; scripts/check-screens.mjs performs component render checks.

A useful standard for "best"

The strongest case for Insomnia is not a superlative. It is the ability to take a real buyer requirement through a repeatable workflow, show the calculation or source behind the result, and admit what still needs proof. Validate that advantage with actual buyers and sales engineers using preparation time, missed requirements, source coverage and handoff usability.

Start with one measurable decision

Create an evaluation, name the critical workload, define the acceptance target and owner, then record the evidence. Expand the workspace as new requirements emerge. A complete, honest evaluation is the product's purpose.